Python Chat App (LAN)

Complete project: report, source code, 2-user test and chat logs

Objective

Create a local real-time chat app.

1. Introduction

Instant messaging is one of the most common uses of computer networks. This project builds a real-time chat application that works over a Local Area Network (LAN) without any internet connection or external service. One computer runs a chat server, and any number of users on the same network connect to it with a graphical client and talk to each other instantly. The project demonstrates sockets, client-server architecture, concurrency and event-driven user interfaces.

2. Abstract

The application consists of a multi-threaded TCP server and a Tkinter-based client. The server accepts many clients at once, giving each its own thread, and broadcasts every message to the other connected users. It announces when users join or leave, rejects duplicate usernames, and records all events with timestamps in a log file. Users can type commands: **/exit** to leave, **/mute** to stop receiving chat messages and **/users** to see who is online. An automated two-user test (Alice and Bob) verified message delivery, muting, join/leave notices and logging.

3. Tools Used

- **Python 3**: programming language for server and client.
- **socket**: TCP communication between server and clients.
- **threading**: one thread per client on the server, a receiver thread in each client.
- **Tkinter** (ScrolledText, Entry, Button, simpledialog): the chat window.
- **queue**: passes received messages safely from the network thread to the GUI.
- **datetime** and file I/O: timestamped logging to *chat_log.txt*.
- **subprocess**: lets the test script start the server automatically.

4. Steps Involved in Building the Project

1. **Set up the socket server.** Created a TCP socket, bound it to port 5000 on all interfaces and started listening for connections.
2. **Allow multiple clients using threads.** Each accepted connection runs *handle_client()* in its own thread. A lock-protected dictionary stores each user's name and mute state.
3. **Create chat input/output UI.** Built a Tkinter window with a read-only scrolling message area and an input box. A background thread reads the socket and a queue feeds the window so the interface never freezes.
4. **Handle join/leave events.** The first line a client sends is its username. The server broadcasts "joined" and "left" notices and refuses duplicate names.
5. **Save logs to file.** *log()* appends every event with a timestamp to *chat_log.txt*, using a lock so threads do not mix lines.
6. **Add command support.** Lines starting with "/" are commands: /exit, /mute (toggle) and /users. Unknown commands get an error reply.
7. **Test with two users.** *test_two_users.py* starts the server, connects Alice and Bob, sends messages, tests /mute and /exit and saves the results.

How to Run

- Start the server: `python server.py` (default port 5000).

- Start each client: `python client.py`. Use IP 127.0.0.1 on the same machine, or the server's LAN IP from other computers.
- Automated 2-user test: `python test_two_users.py`.

5. Conclusion

The project delivers a working LAN chat system with multi-user support, a graphical interface, event handling, logging and command support. It gave practical experience with sockets, threads, thread-safe data sharing and GUI programming. Future improvements could include private messages, encryption (TLS), file sharing, user authentication and a chat history viewer.

Deliverable 1: Server Code (server.py)

```
"""LAN Chat Server - socket + threading.

Run:  python server.py [port]
Each client gets its own thread. All chat events are written to chat_log.txt.
Protocol: plain UTF-8 text, one message per line.
  - First line from a client is its username.
  - Lines starting with "/" are commands (/exit, /mute, /users).
"""
import socket
import sys
import threading
from datetime import datetime

HOST = "0.0.0.0"          # listen on all LAN interfaces
PORT = 5000
LOG_FILE = "chat_log.txt"

clients = {}              # socket -> {"name": str, "muted": bool}
clients_lock = threading.Lock()
log_lock = threading.Lock()


def log(text):
    """Print and append a timestamped line to the log file."""
    line = f"[{datetime.now():%Y-%m-%d %H:%M:%S}] {text}"
    print(line)
    with log_lock:
        with open(LOG_FILE, "a", encoding="utf-8") as f:
            f.write(line + "\n")


def send(sock, text):
    try:
        sock.sendall((text + "\n").encode("utf-8"))
    except OSError:
        pass


def broadcast(text, sender=None, system=False):
    """Send text to everyone except the sender and muted clients.
    System messages (join/leave) are always delivered."""
    with clients_lock:
        targets = list(clients.items())
    for sock, info in targets:
        if sock is sender:
            continue
        if info["muted"] and not system:
            continue
        send(sock, text)


def name_taken(name):
    with clients_lock:
        return any(i["name"].lower() == name.lower() for i in clients.values())


def remove_client(sock):
    with clients_lock:
        info = clients.pop(sock, None)
    try:
        sock.close()
    except OSError:
        pass
    return info


def handle_client(sock, addr):
    reader = sock.makefile("r", encoding="utf-8", newline="\n")
    name = None
    try:
        name = (reader.readline() or "").strip()
        if not name:
```

```
            return
        if name_taken(name):
            send(sock, "SYSTEM: That name is already in use. Reconnect with another name.")
            return

        with clients_lock:
            clients[sock] = {"name": name, "muted": False}
        log(f"JOIN: {name} connected from {addr[0]}:{addr[1]}")
        send(sock, f"SYSTEM: Welcome, {name}! Commands: /exit, /mute, /users")
        broadcast(f"SYSTEM: {name} joined the chat", sender=sock, system=True)

        for line in reader:
            msg = line.strip()
            if not msg:
                continue

            if msg.startswith("/"):
                cmd = msg.lower()
                if cmd == "/exit":
                    send(sock, "SYSTEM: Goodbye!")
                    break
                elif cmd == "/mute":
                    with clients_lock:
                        clients[sock]["muted"] = not clients[sock]["muted"]
                        muted = clients[sock]["muted"]
                    state = "ON (you won't receive chat messages)" if muted else "OFF"
                    send(sock, f"SYSTEM: Mute {state}")
                    log(f"CMD: {name} -> /mute {'ON' if muted else 'OFF'}")
                elif cmd == "/users":
                    with clients_lock:
                        names = ", ".join(i["name"] for i in clients.values())
                    send(sock, f"SYSTEM: Online: {names}")
                else:
                    send(sock, f"SYSTEM: Unknown command {msg}")
                continue

            log(f"MSG: {name}: {msg}")
            broadcast(f"{name}: {msg}", sender=sock)
    except (ConnectionError, OSError):
        pass
    finally:
        if remove_client(sock) and name:
            log(f"LEAVE: {name} disconnected")
            broadcast(f"SYSTEM: {name} left the chat", system=True)


def main():
    port = int(sys.argv[1]) if len(sys.argv) > 1 else PORT
    server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    server.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
    server.bind((HOST, port))
    server.listen()
    log(f"SERVER: listening on {HOST}:{port}")
    try:
        while True:
            sock, addr = server.accept()
            threading.Thread(target=handle_client, args=(sock, addr), daemon=True).start()
    except KeyboardInterrupt:
        log("SERVER: shutting down")
    finally:
        server.close()


if __name__ == "__main__":
    main()
```

Deliverable 2: Client Code (client.py)

```
"""LAN Chat Client - Tkinter GUI.

Run:  python client.py
Enter the server IP (use 127.0.0.1 on the same machine), port and your name.
Commands: /exit  /mute  /users
"""
import queue
import socket
import threading
import tkinter as tk
from tkinter import messagebox, scrolledtext, simpledialog

DEFAULT_HOST = "127.0.0.1"
DEFAULT_PORT = 5000


class ChatClient:
    def __init__(self, root, sock, name):
        self.root = root
        self.sock = sock
        self.name = name
        self.incoming = queue.Queue()
        self.closed = False

        root.title(f"LAN Chat - {name}")
        root.geometry("480x520")

        self.chat = scrolledtext.ScrolledText(root, state="disabled", wrap="word")
        self.chat.pack(fill="both", expand=True, padx=8, pady=(8, 4))
        self.chat.tag_config("system", foreground="#888888")
        self.chat.tag_config("me", foreground="#1a5fb4")

        bottom = tk.Frame(root)
        bottom.pack(fill="x", padx=8, pady=(0, 8))
        self.entry = tk.Entry(bottom)
        self.entry.pack(side="left", fill="x", expand=True)
        self.entry.bind("<Return>", self.send_message)
        self.entry.focus()
        tk.Button(bottom, text="Send", command=self.send_message).pack(side="left", padx=(6, 0))

        root.protocol("WM_DELETE_WINDOW", self.close)
        threading.Thread(target=self.receive_loop, daemon=True).start()
        self.root.after(100, self.process_incoming)

    # ---- networking ----
    def receive_loop(self):
        try:
            reader = self.sock.makefile("r", encoding="utf-8", newline="\n")
            for line in reader:
                self.incoming.put(line.rstrip("\n"))
        except OSError:
            pass
        self.incoming.put(None)  # connection closed

    def send_message(self, event=None):
        text = self.entry.get().strip()
        if not text:
            return
        self.entry.delete(0, "end")
        try:
            self.sock.sendall((text + "\n").encode("utf-8"))
        except OSError:
            self.show("SYSTEM: Connection lost.", "system")
            return
        if text.lower() == "/exit":
            self.root.after(300, self.close)
        elif not text.startswith("/"):
            self.show(f"You: {text}", "me")

    # ---- UI ----
    def show(self, text, tag=None):
        self.chat.config(state="normal")
```

```
        self.chat.insert("end", text + "\n", tag)
        self.chat.config(state="disabled")
        self.chat.see("end")

    def process_incoming(self):
        try:
            while True:
                msg = self.incoming.get_nowait()
                if msg is None:
                    self.show("SYSTEM: Disconnected from server.", "system")
                    self.entry.config(state="disabled")
                    return
                self.show(msg, "system" if msg.startswith("SYSTEM:") else None)
        except queue.Empty:
            pass
        if not self.closed:
            self.root.after(100, self.process_incoming)

    def close(self):
        self.closed = True
        try:
            self.sock.close()
        except OSError:
            pass
        self.root.destroy()


def main():
    root = tk.Tk()
    root.withdraw()
    host = simpledialog.askstring("Server", "Server IP:", initialvalue=DEFAULT_HOST, parent=root)
    if not host:
        return
    port = simpledialog.askinteger("Server", "Port:", initialvalue=DEFAULT_PORT, parent=root)
    name = simpledialog.askstring("Name", "Your name:", parent=root)
    if not port or not name or not name.strip():
        return
    name = name.strip()

    try:
        sock = socket.create_connection((host, port), timeout=5)
        sock.settimeout(None)
        sock.sendall((name + "\n").encode("utf-8"))
    except OSError as e:
        messagebox.showerror("Connection failed", str(e))
        return

    root.deiconify()
    ChatClient(root, sock, name)
    root.mainloop()


if __name__ == "__main__":
    main()
```

Deliverable 3: 2-User Test Script (test_two_users.py)

```
"""Automated 2-user test (no GUI needed).

Starts the server, connects Alice and Bob over real sockets, runs a scripted
conversation (including /mute and /exit), and saves what each user saw to
test_output.txt. The server writes chat_log.txt.
"""
import os
import socket
import subprocess
import sys
import time

PORT = 5055
HERE = os.path.dirname(os.path.abspath(__file__))


class Client:
    def __init__(self, name):
        self.name = name
        self.sock = socket.create_connection(("127.0.0.1", PORT), timeout=3)
        self.sock.sendall((name + "\n").encode())
        self.seen = []

    def say(self, text):
        self.sock.sendall((text + "\n").encode())
        time.sleep(0.3)

    def read(self):
        self.sock.settimeout(0.3)
        try:
            data = self.sock.recv(65536).decode()
            self.seen += [l for l in data.splitlines() if l]
        except (socket.timeout, OSError):
            pass


def main():
    os.chdir(HERE)
    if os.path.exists("chat_log.txt"):
        os.remove("chat_log.txt")
    server = subprocess.Popen([sys.executable, "server.py", str(PORT)],
                              stdout=subprocess.DEVNULL)
    time.sleep(1)
    try:
        alice = Client("Alice"); time.sleep(0.3)
        bob = Client("Bob"); time.sleep(0.3)

        alice.say("Hi Bob!")
        bob.say("Hey Alice, how are you?")
        alice.say("/users")
        bob.say("/mute")                        # Bob mutes
        alice.say("Can you still see this, Bob?")
        bob.say("/mute")                        # Bob unmutes
        alice.say("Welcome back!")
        bob.say("Thanks! Bye.")
        bob.say("/exit")
        time.sleep(0.3)
        alice.say("/exit")

        for c in (alice, bob):
            c.read()
    finally:
        time.sleep(0.3)
        server.terminate()

    with open("test_output.txt", "w", encoding="utf-8") as f:
        for c in (alice, bob):
            f.write(f"=== What {c.name} saw ===\n" + "\n".join(c.seen) + "\n\n")
    print(open("test_output.txt", encoding="utf-8").read())
    print("=== chat_log.txt ===")
    print(open("chat_log.txt", encoding="utf-8").read())
if __name__ == "__main__":
```

```
main()
```

Deliverable 4: 2-User Test Output (test_output.txt)

What each user (Alice and Bob) saw during the automated test. Note that Bob did not receive Alice's message "Can you still see this, Bob?" because he had muted chat.

```
=== What Alice saw ===
SYSTEM: Welcome, Alice! Commands: /exit, /mute, /users
SYSTEM: Bob joined the chat
Bob: Hey Alice, how are you?
SYSTEM: Online: Alice, Bob
Bob: Thanks! Bye.
SYSTEM: Bob left the chat
SYSTEM: Goodbye!

=== What Bob saw ===
SYSTEM: Welcome, Bob! Commands: /exit, /mute, /users
Alice: Hi Bob!
SYSTEM: Mute ON (you won't receive chat messages)
SYSTEM: Mute OFF
Alice: Welcome back!
SYSTEM: Goodbye!
```

Deliverable 5: Chat Logs (chat_log.txt)

Timestamped server log of the same test session.

```
[2026-11-06 08:15:59] SERVER: listening on 0.0.0.0:5055
[2026-11-06 08:16:00] JOIN: Alice connected from 127.0.0.1:56682
[2026-11-06 08:16:00] JOIN: Bob connected from 127.0.0.1:56688
[2026-11-06 08:16:00] MSG: Alice: Hi Bob!
[2026-11-06 08:16:01] MSG: Bob: Hey Alice, how are you?
[2026-11-06 08:16:01] CMD: Bob -> /mute ON
[2026-11-06 08:16:02] MSG: Alice: Can you still see this, Bob?
[2026-11-06 08:16:02] CMD: Bob -> /mute OFF
[2026-11-06 08:16:02] MSG: Alice: Welcome back!
[2026-11-06 08:16:03] MSG: Bob: Thanks! Bye.
[2026-11-06 08:16:03] LEAVE: Bob disconnected
[2026-11-06 08:16:03] LEAVE: Alice disconnected
```